

CI/CD Deploy on 3-Tier Architecture (Kubernetes)

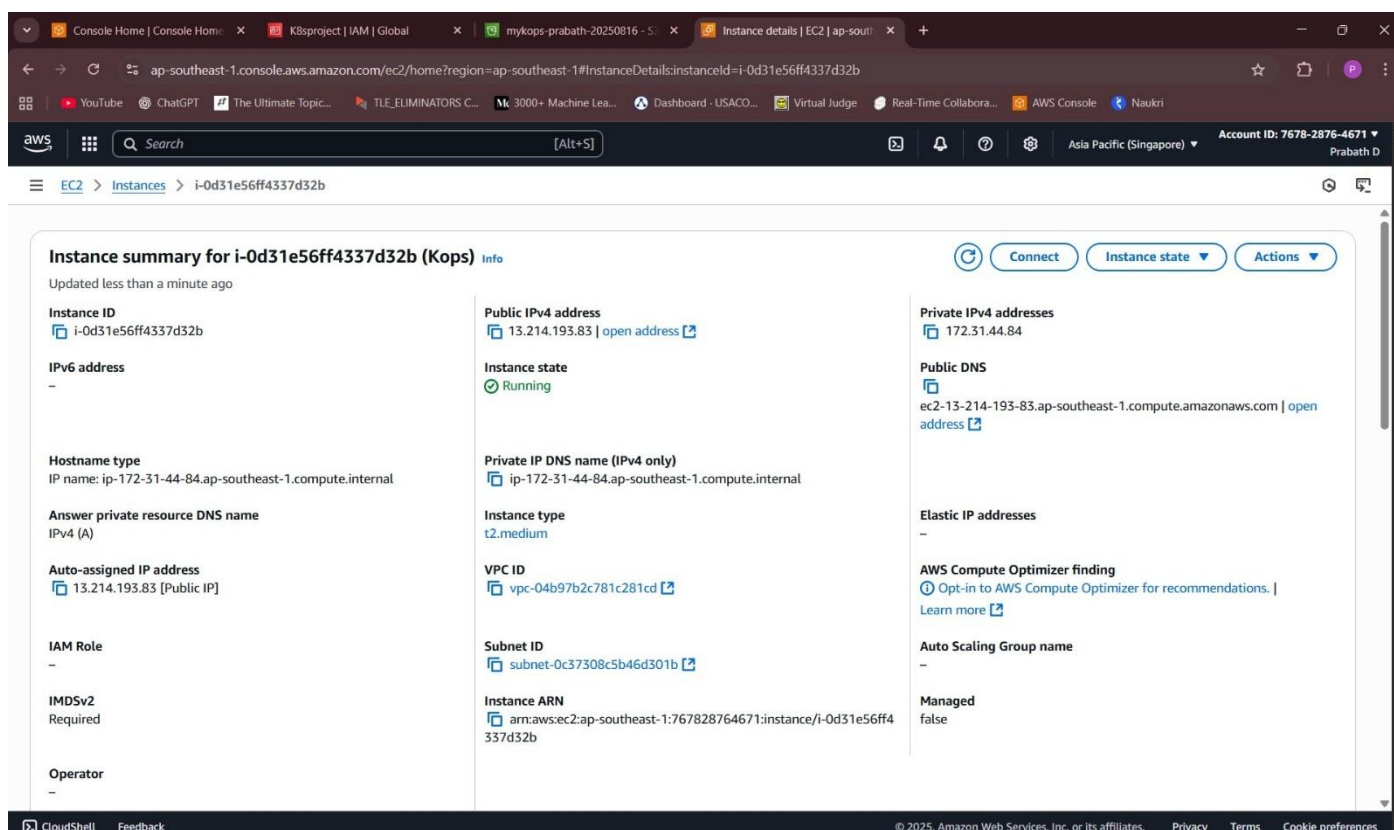
Objective:

The objective is to build and deploy CI/CD Pipeline on 3-tier architecture through Kubernetes. The tools used in this project are GitHub, Jenkins, Kubernetes and Docker. The expected output is to show the httpd image in the docker repository and Student Registration Form page.

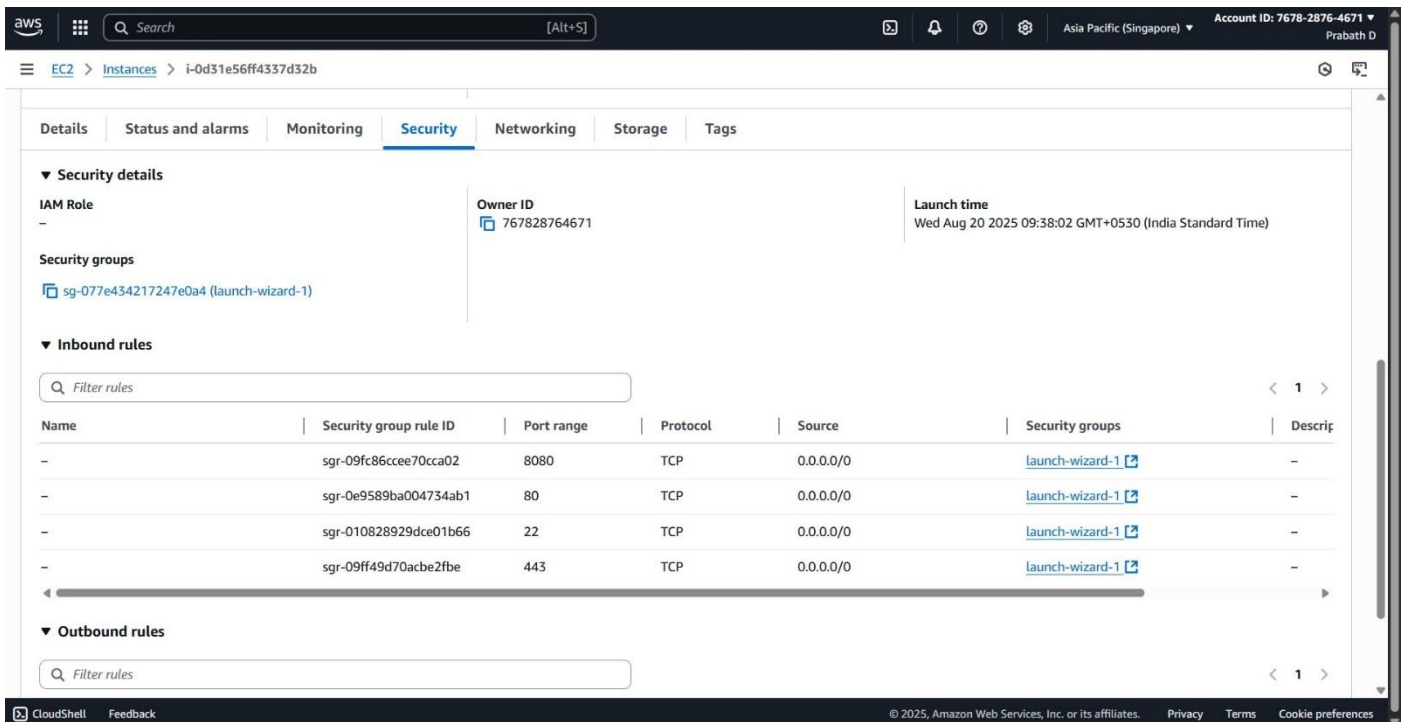
Steps:

i. Setup Instance:

- Go to AWS Console -> EC2 Instance -> Launch Instance
- Set the name as 'Kops', Select AMI Server as 'Ubuntu', then select key pair.
- Then in Network Settings, click existing security group which must contain HTTPS, HTTP, SSH, and Custom TCP port number 8080 (for Jenkins).
- Then launch instance.



(Kops EC2 Instance)



(Security Groups)

ii. Jenkins Installation:

- Copy and paste the public IP address of Kops Instance and open the EC2 SSH Terminal (Putty).
- Then give the commands one by one for installation of Jenkins.
- Before installing Jenkins, Java SDE must be installed in the putty server.
- Visit Jenkins.io website, then click Linux to install the Jenkins, then scroll down there the code for java installation will be available.

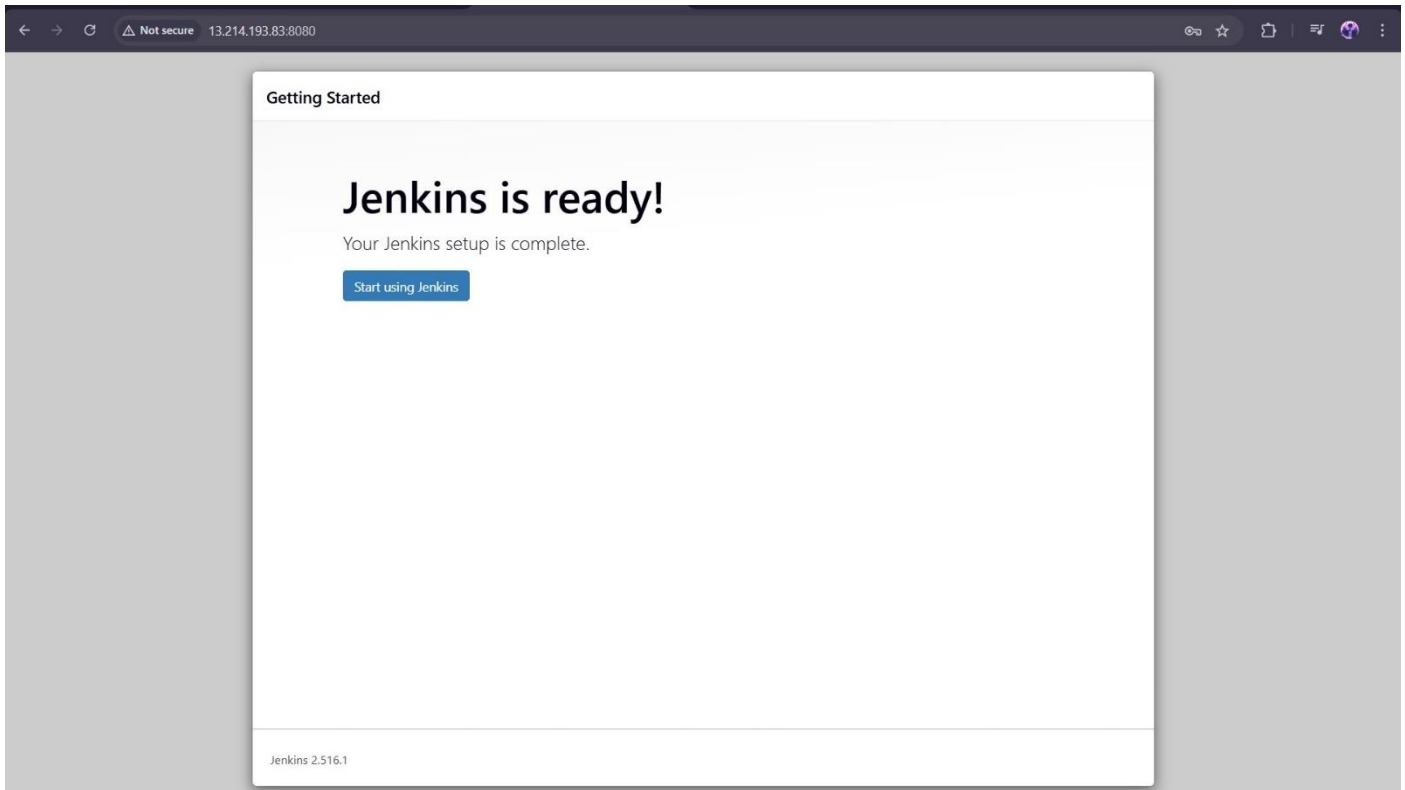
→ Java Code:

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
```

→ Jenkins Installation Code:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install Jenkins
```

- Copy and paste the java code, after the installation is complete, paste the Jenkins installation code.
- Then open the Jenkins server by copy and pasting the IP address of 'Kops' and hit it with <IP Address:8080>, this is how the Jenkins server runs.
- It will ask for administrator password copy the directory and paste it in the putty by giving the "cat <directory>" it will give one password, then setup the Jenkins by installing the Plugins.



iii. Create Kubernetes Cluster:

- First, the binary setup has to be done for Kubernetes so the resources and packages must be installed, then the cluster need to be created.
- As a whole, the code is created for binary setup and cluster creation.
- After installation of the binary setup, and the creation of cluster it needs to be configured.

→ Code:

```
curl -Lo kops https://github.com/kubernetes/kops/releases/download/$(curl -s
https://api.github.com/repos/kubernetes/kops/releases/latest | grep tag_name | cut -d
'"' -f 4)/kops-linux-amd64
chmod +x ./kops
sudo mv ./kops /usr/local/bin/kops
```

```

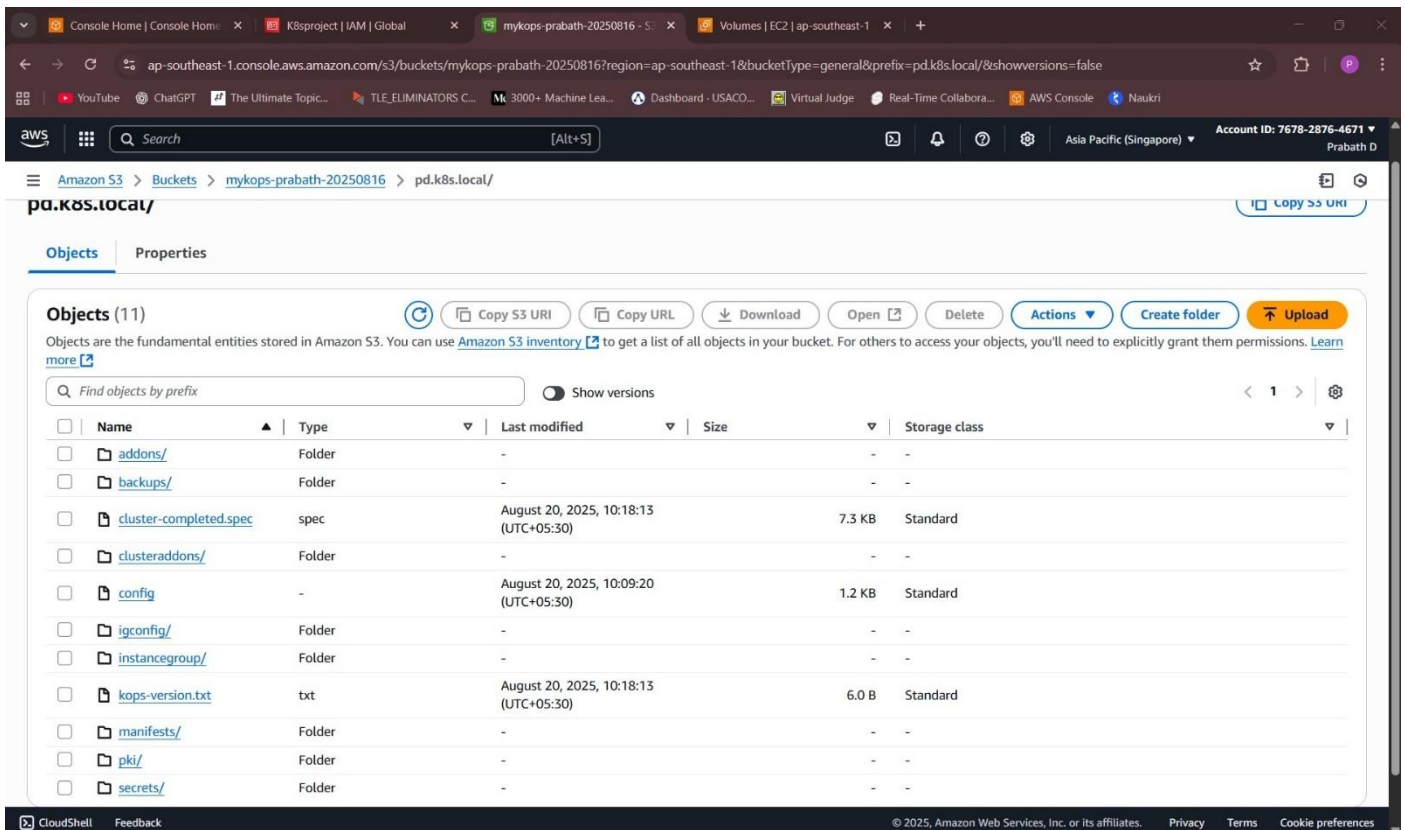
curl -Lo kubectl https://storage.googleapis.com/kubernetes-release/release/$(curl -s
https://storage.googleapis.com/kubernetes-
release/release/stable.txt)/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin/kubectl
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws s3api create-bucket \
  --bucket mykops-prabath-20250816 \
  --region ap-southeast-1 \
  --create-bucket-configuration LocationConstraint=ap-southeast-1
aws s3api put-bucket-versioning \
  --bucket newkops8-prabath-2025 \
  --versioning-configuration Status=Enabled
export KOPS_STATE_STORE=s3://mykops-prabath-20250816
export NAME=pd.k8s.local

```

- So, the S3 Bucket will be created in the AWS Console automatically, but before that one IAM User need to created for access key ID and password,
- Then set the default region, and language used to code.
- By giving the credentials, the cluster will be created successfully.

The screenshot displays the AWS IAM console interface for a user named 'K8sproject'. The user's ARN is 'arn:aws:iam::767828764671:user/K8sproject'. It shows two active access keys: 'AKIA3FRJE774VDBYVHE' (used 20 hours ago) and 'AKIA3FRJE775PZPUVX6' (used 11 hours ago). Under the 'Permissions' tab, a table lists one policy: 'AdministratorAccess', which is an AWS managed policy attached directly. The interface includes navigation links for Summary, Groups, Tags, Security credentials, and Last Accessed. At the bottom, there is a section for 'Permissions boundary' which is currently not set.

IAM-User



S3-Bucket

iv. Create Directories and Files:

- After the creation of cluster, the files need to be created and saved in the form of directory.

```

project/
├── backend/
│   ├── Dockerfile
│   └── src/ app.py requirements.txt
├── frontend/
│   ├── Dockerfile
│   ├── index.html student.html backend.html
└── k8s/
    ├── backend-deployment.yaml
    ├── frontend-deployment.yaml
    └── db-deployment.yaml
  
```

- In this format, the directories are need to be pushed into GitHub.
- Create a directory named with “myproject”, inside that directory make those folders and files present.

→ Codes:

▪ Project Folder:

```
mkdir project
cd project
mkdir backend
cd backend
touch Dockerfile
mkdir src
cd src
touch app.py requirements.txt
```

app.py:

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import mysql.connector
import bcrypt
import os
```

```
app = Flask(__name__)
CORS(app)
```

```
conn = mysql.connector.connect(
    host=os.environ['DB_HOST'],
    user=os.environ['DB_USER'],
    password=os.environ['DB_PASSWORD'],
    database=os.environ['DB_NAME']
)
cursor = conn.cursor()
```

```
@app.route('/signup', methods=['POST'])
def signup():
    data = request.json
```

```

        hashed_pw = bcrypt.hashpw(data['password'].encode('utf-8'), bcrypt.gensalt())

        cursor.execute("INSERT INTO users (first_name, middle_name, last_name, username,
password, email) VALUES (%s, %s, %s, %s, %s, %s)",
                        (data['first_name'], data['middle_name'], data['last_name'], data['username'],
hashed_pw, data['email']))

        conn.commit()

        return jsonify({'msg': 'User registered successfully'})

```

```

@app.route('/login', methods=['POST'])

```

```

def login():

```

```

    data = request.json

    cursor.execute("SELECT password FROM users WHERE username = %s",
(data['username'],))

    result = cursor.fetchone()

    if result and bcrypt.checkpw(data['password'].encode('utf-8'), result[0].encode('utf-8')):
        return jsonify({'msg': 'Login successful'})

    return jsonify({'msg': 'Invalid credentials'}), 401

```

```

@app.route('/')

```

```

def home():

```

```

    return 'Welcome to the Knowledge Acquisition API'

```

```

@app.route('/student', methods=['POST'])

```

```

def student():

```

```

    data = request.json

    cursor = conn.cursor()

    cursor.execute("INSERT INTO students (name, date_of_joining, fees_paid, department,
trainer_name, company_name) VALUES (%s, %s, %s, %s, %s, %s)",
                    (data['name'], data['date_of_joining'], data['fees_paid'], data['department'],
data['trainer_name'], data['company_name']))

    conn.commit()

```

```

    cursor.close()

    return jsonify({'msg': 'Student data stored'})

@app.route('/students', methods=['GET'])
def get_students():
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM students") # Use your actual students table name
    students = cursor.fetchall()
    cursor.close()
    return jsonify(students)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

requirements.txt:

```

flask
flask-cors
bcrypt
mysql-connector-python

```

Backend/ Dockerfile:

```

FROM python:3.11-slim
WORKDIR /app
COPY src/requirements.txt .
RUN pip install -r requirements.txt
COPY src/ .
CMD ["python", "app.py"]

```

- Add these codes in the necessary files then save it. After that use `cd ..` for exiting the current folder.

➤ Then do the same for frontend and k8s folder.

→ **Frontend Folder:**

```
cd project
mkdir frontend
cd frontend
touch Dockerfile
touch index.html backend.html student.html
```

index.html:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1" />

<title>Knowledge Acquisition</title>

<style>

* { box-sizing: border-box; }

body {

font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

background: #f0f4f8;

margin: 0; padding: 0;

display: flex; flex-direction: column;

min-height: 100vh;

justify-content: center; align-items: center;

text-align: center; color: #2c3e50;

}

h1 {
```

```
font-size: 2.5rem; margin-bottom: 30px;
}

.form-container {
  background: white;
  padding: 30px 40px;
  border-radius: 10px;
  box-shadow: 0 8px 16px rgba(0,0,0,0.1);
  width: 350px; max-width: 90%;
}

.toggle-buttons {
  margin-bottom: 20px;
  display: flex; justify-content: center; gap: 15px;
}

.toggle-buttons button {
  flex: 1;
  padding: 10px 0;
  font-size: 1rem;
  border-radius: 25px;
  border: 2px solid #3498db;
  background-color: white;
  color: #3498db;
  cursor: pointer;
  transition: all 0.3s ease;
}

.toggle-buttons button.active, .toggle-buttons button:hover {
  background-color: #3498db;
```

```
color: white;

}

form {

  display: none;

  flex-direction: column;

  gap: 15px;

}

form.active {

  display: flex;

}

input[type="text"], input[type="email"], input[type="password"] {

  padding: 10px;

  font-size: 1rem;

  border: 1.5px solid #ccc;

  border-radius: 6px;

  outline-color: #3498db;

  transition: border-color 0.3s ease;

}

input[type="text"]:focus, input[type="email"]:focus, input[type="password"]:focus {

  border-color: #3498db;

}

button.submit-btn {

  background-color: #3498db;

  border: none;

  color: white;

  padding: 12px 0;
```

```
font-size: 1.1rem;

border-radius: 8px;

cursor: pointer;

transition: background-color 0.3s ease;

}

button.submit-btn:hover {

background-color: #2980b9;

}

@media (max-width: 480px) {

h1 {

font-size: 1.8rem;

margin-bottom: 20px;

}

.form-container {

width: 90%;

padding: 20px;

}

.toggle-buttons button {

font-size: 0.9rem;

padding: 8px 0;

}

}

</style>

</head>

<body>

<h1>Welcome to Greens Tech Academy</h1>
```

```
<div class="form-container">
```

```
  <div class="toggle-buttons">
```

```
    <button id="signin-btn" class="active">Sign In</button>
```

```
    <button id="signup-btn">Sign Up</button>
```

```
  </div>
```

```
<!-- Sign In Form -->
```

```
<form id="signin-form" class="active" onsubmit="login(event)">
```

```
  <input type="text" id="signin-username" placeholder="Username" required />
```

```
  <input type="password" id="signin-password" placeholder="Password" required />
```

```
  <button type="submit" class="submit-btn">Sign In</button>
```

```
</form>
```

```
<!-- Sign Up Form -->
```

```
<form id="signup-form" onsubmit="signup(event)">
```

```
  <input type="text" id="signup-first_name" placeholder="First Name" required />
```

```
  <input type="text" id="signup-middle_name" placeholder="Middle Name" />
```

```
  <input type="text" id="signup-last_name" placeholder="Last Name" required />
```

```
  <input type="text" id="signup-username" placeholder="Username" required />
```

```
  <input type="password" id="signup-password" placeholder="Password" required />
```

```
  <input type="email" id="signup-email" placeholder="Email" required />
```

```
  <button type="submit" class="submit-btn">Sign Up</button>
```

```
</form>
```

```
</div>
```

```
<script>
```

```
const signinBtn = document.getElementById('signin-btn');
```

```
const signupBtn = document.getElementById('signup-btn');
```

```
const signinForm = document.getElementById('signin-form');
```

```
const signupForm = document.getElementById('signup-form');
```

```
signinBtn.addEventListener('click', () => {
```

```
  signinBtn.classList.add('active');
```

```
  signupBtn.classList.remove('active');
```

```
  signinForm.classList.add('active');
```

```
  signupForm.classList.remove('active');
```

```
});
```

```
signupBtn.addEventListener('click', () => {
```

```
  signupBtn.classList.add('active');
```

```
  signinBtn.classList.remove('active');
```

```
  signupForm.classList.add('active');
```

```
  signinForm.classList.remove('active');
```

```
});
```

```
async function signup(e) {
```

```
  e.preventDefault();
```

```
  const res = await fetch('http://a23805e4dd81246caab4cd122c0c0567-1998564976.ap-  
south-1.elb.amazonaws.com:5000/signup', {
```

```
    method: 'POST',
```

```
    headers: {'Content-Type': 'application/json'},
```

```

body: JSON.stringify({
    first_name: document.getElementById('signup-first_name').value,
    middle_name: document.getElementById('signup-middle_name').value,
    last_name: document.getElementById('signup-last_name').value,
    username: document.getElementById('signup-username').value,
    password: document.getElementById('signup-password').value,
    email: document.getElementById('signup-email').value
}))
});

const data = await res.json();

alert(data.msg);

if(res.ok) {
    // After successful signup, switch to sign in form
    signinBtn.click();
}
}

async function login(e) {
    e.preventDefault();

    const res = await fetch('http://a23805e4dd81246caab4cd122c0c0567-1998564976.ap-
south-1.elb.amazonaws.com:5000/login', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            username: document.getElementById('signin-username').value,
            password: document.getElementById('signin-password').value

```

```
    })

});

const data = await res.json();

alert(data.msg);

if(res.ok) {

    // Redirect to backend page after successful login

    window.location.href = 'backend.html';

}

}

</script>

</body>

</html>
```

backend.html:

```
<h2>Department: DevOps</h2>

<form onsubmit="saveStudent(event)" class="student-form">

    <input placeholder="Name" id="name" required />

    <input type="date" id="date" required />

    <input placeholder="Fees Paid" id="fees" required />

    <input placeholder="Trainer Name" id="trainer" required />

    <input placeholder="Company Name" id="company" required />

    <input value="DevOps" id="department" readonly />

    <button type="submit">Submit</button>

</form>

<style>

.student-form {

    max-width: 400px;
```



```
margin: 0 auto 40px auto;
background: #fff;
padding: 25px 30px;
border-radius: 10px;
box-shadow: 0 8px 16px rgba(0,0,0,0.1);
display: flex;
flex-direction: column;
gap: 15px;
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
```

```
.student-form input {
padding: 10px 15px;
font-size: 1rem;
border: 1.5px solid #ccc;
border-radius: 6px;
outline-color: #3498db;
transition: border-color 0.3s ease;
}
```

```
.student-form input:focus {
border-color: #3498db;
}
```

```
.student-form input[readonly] {
background-color: #e9ecef;
cursor: not-allowed;
}
```

```
.student-form button {
padding: 12px 0;
font-size: 1.1rem;
```

```
border: none;
border-radius: 8px;
background-color: #3498db;
color: white;
cursor: pointer;
transition: background-color 0.3s ease;
}
```

```
.student-form button:hover {
  background-color: #2980b9;
}
```

```
h2 {
  text-align: center;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  margin-bottom: 20px;
  color: #2c3e50;
}
```

```
@media (max-width: 480px) {
  .student-form {
    padding: 20px;
    max-width: 90%;
    gap: 12px;
  }
  .student-form input, .student-form button {
    font-size: 1rem;
    padding: 10px 12px;
  }
}
```

```
</style>
```

```

<script>
  async function saveStudent(e) {
    e.preventDefault();

    const res = await fetch('http://a23805e4dd81246caab4cd122c0c0567-1998564976.ap-
south-1.elb.amazonaws.com:5000/student', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        name: document.getElementById('name').value,
        date_of_joining: document.getElementById('date').value,
        fees_paid: document.getElementById('fees').value,
        department: document.getElementById('department').value,
        trainer_name: document.getElementById('trainer').value,
        company_name: document.getElementById('company').value
      })
    });

    const data = await res.json();
    alert(data.msg);
    if (res.ok) {
      window.location.href = 'student.html';
    }
  }
</script>

```

student.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />

```

```
<title>Students List</title>
```

```
<style>
```

```
body {
```

```
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
    max-width: 900px;
```

```
    margin: 40px auto;
```

```
    padding: 10px;
```

```
    background: #f9f9f9;
```

```
}
```

```
h2 {
```

```
    text-align: center;
```

```
    color: #2c3e50;
```

```
}
```

```
table {
```

```
    width: 100%;
```

```
    border-collapse: collapse;
```

```
    margin-top: 20px;
```

```
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
```

```
}
```

```
th, td {
```

```
    padding: 12px;
```

```
    border: 1px solid #ddd;
```

```
    text-align: left;
```

```
}
```

```
th {
```

```
    background-color: #3498db;
```

```
    color: white;
```

```
}
```

```
tr:nth-child(even) {
```

```
    background-color: #f2f2f2;
```

```
}  
@media (max-width: 600px) {  
  table, thead, tbody, th, td, tr {  
    display: block;  
  }  
  th {  
    position: absolute;  
    top: -9999px;  
    left: -9999px;  
  }  
  tr {  
    margin-bottom: 15px;  
  }  
  td {  
    border: none;  
    position: relative;  
    padding-left: 50%;  
    text-align: right;  
  }  
  td::before {  
    position: absolute;  
    left: 15px;  
    width: 45%;  
    white-space: nowrap;  
    content: attr(data-label);  
    font-weight: bold;  
    text-align: left;  
  }  
}  
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Students Registered</h2>
```

```
<table id="students-table">
```

```
<thead>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Date of Joining</th>
```

```
<th>Fees Paid</th>
```

```
<th>Department</th>
```

```
<th>Trainer Name</th>
```

```
<th>Company Name</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<!-- Students data will be inserted here -->
```

```
</tbody>
```

```
</table>
```

```
<script>
```

```
async function loadStudents() {
```

```
  try {
```

```
    const res = await fetch('http://a23805e4dd81246caab4cd122c0c0567-1998564976.ap-south-1.elb.amazonaws.com:5000/students'); // Your backend should expose this API
```

```
    if (!res.ok) throw new Error('Failed to fetch students');
```

```
    const students = await res.json();
```

```
    const tbody = document.querySelector('#students-table tbody');
```

```
    tbody.innerHTML = ""; // Clear existing rows
```

```

students.forEach(student => {
    const tr = document.createElement('tr');

    tr.innerHTML = `
        <td data-label="Name">${student.name}</td>
        <td data-label="Date of Joining">${new
Date(student.date_of_joining).toLocaleDateString()}</td>
        <td data-label="Fees Paid">${student.fees_paid}</td>
        <td data-label="Department">${student.department}</td>
        <td data-label="Trainer Name">${student.trainer_name}</td>
        <td data-label="Company Name">${student.company_name}</td>
    `;

    tbody.appendChild(tr);
});
} catch (error) {
    console.error('Error loading students:', error);
    alert('Error loading students. Check console.');
```

frontend/Dockerfile:

```

FROM nginx:alpine
COPY . /usr/share/nginx/html
```

➤ After successful creation of frontend folder, create k8s folder and add the files inside it.

▪ **K8S Project:**

```
cd project
mkdir
cd frontend
touch Dockerfile
touch backend-deployment.yaml frontend-deployment.yaml db-deployment.yaml
```

frontend-deployment.yaml:

apiVersion: apps/v1

kind: Deployment

metadata:

name: frontend

spec:

replicas: 1

selector:

matchLabels:

app: frontend

template:

metadata:

labels:

app: frontend

spec:

containers:

- name: frontend

image: prabathd/frontend-app:latest

ports:

- containerPort: 80

apiVersion: v1


```
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
```

backend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: prabathd/backend-api:latest
          ports:
```

- containerPort: 5000

env:

- name: DB_HOST

 - value: "mysql"

- name: DB_USER

 - value: "root"

- name: DB_PASSWORD

 - value: "rootpassword"

- name: DB_NAME

 - value: "knowledge_acquisition"

apiVersion: v1

kind: Service

metadata:

- name: backend-service

spec:

- type: LoadBalancer # or NodePort for testing

selector:

- app: backend

ports:

- protocol: TCP

 - port: 5000

 - targetPort: 5000

db-deployment.yaml (kind:deployment):

apiVersion: v1

kind: ConfigMap

metadata:

- name: mysql-initdb-config

data:

init.sql: |

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(100),  
  middle_name VARCHAR(100),  
  last_name VARCHAR(100),  
  username VARCHAR(100) UNIQUE,  
  password VARCHAR(255),  
  email VARCHAR(100)  
);
```

```
CREATE TABLE IF NOT EXISTS students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255),  
  date_of_joining DATE,  
  fees_paid DECIMAL(10,2),  
  department VARCHAR(100),  
  trainer_name VARCHAR(100),  
  company_name VARCHAR(100)  
);
```

apiVersion: apps/v1

kind: Deployment

metadata:

name: mysql

spec:

selector:

matchLabels:

app: mysql

replicas: 1

template:

metadata:

labels:

app: mysql

spec:

containers:

- name: mysql

image: mysql:5.7

env:

- name: MYSQL_ROOT_PASSWORD

value: rootpassword

- name: MYSQL_DATABASE

value: knowledge_acquisition

ports:

- containerPort: 3306

volumeMounts:

- name: initdb

mountPath: /docker-entrypoint-initdb.d

volumes:

- name: initdb

configMap:

name: mysql-initdb-config

apiVersion: v1

kind: Service

metadata:

name: mysql

spec:

type: ClusterIP

selector:

app: mysql

ports:

- port: 3306

targetPort: 3306

- Now, all the files and folders are created. This need to be pushed inside GitHub. But before that docker need to be installed in the putty server.

→ **Code for Docker Installation: apt install docker.io -y**

v. GitHub Repo:

- Create a repository named “k8s-3tier-arch” and set it as public in git hub.
- Then copy the git remote origin “git-project-username-url”.
- In the putty server, run these commands to push the project folder inside the git hub.

→ **Commands:**

git add .

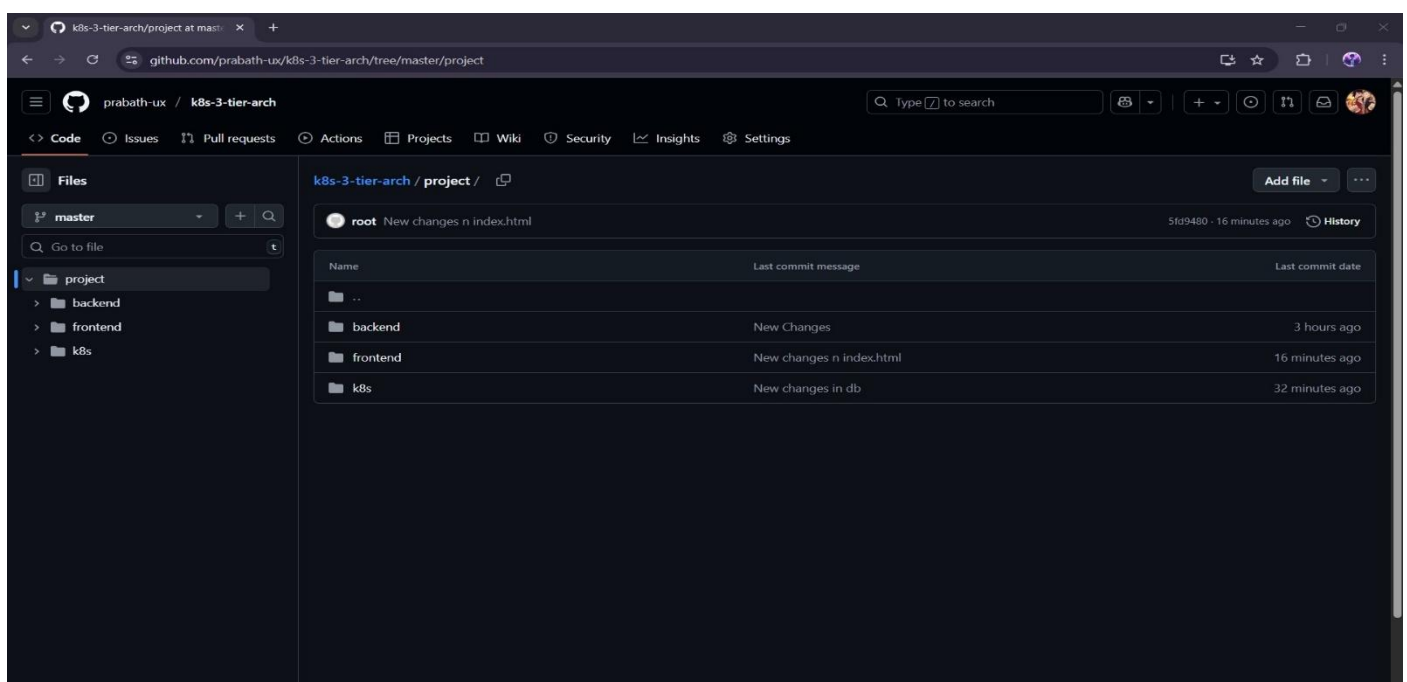
git commit -m “New Changes”

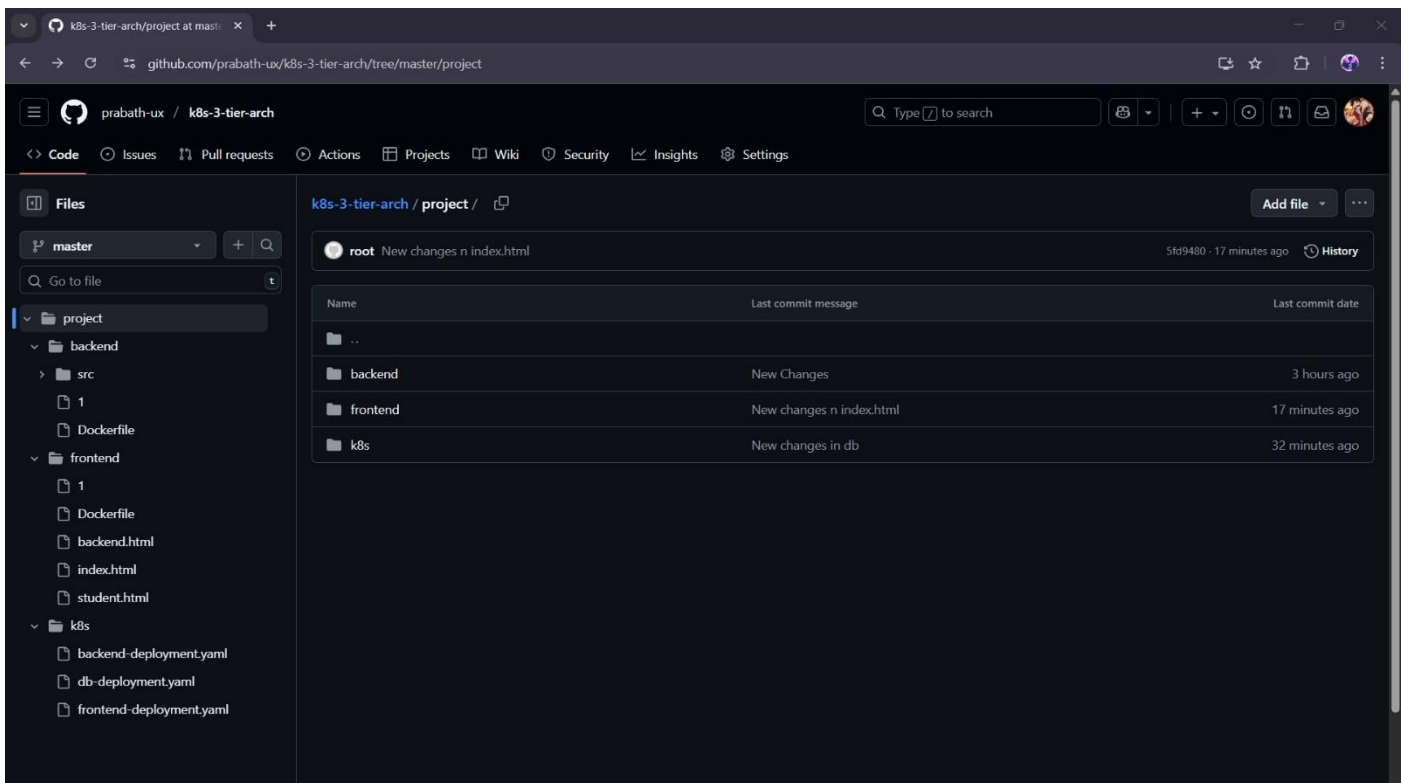
then paste the git remote url.

git remote origin “git-project-username-url”

git push origin -u master

- It will ask for git hub username and password. For password create Personal Access Token in Git hub and paste it in the password place. Then give enter, the project folder with all the files will be pushed inside the GitHub.





➤ Now, let's initialize the Jenkins automation for Kubernetes pods that would reflect in Docker repository in the creation of Backend and Frontend images for 3-Tier Application.

vi. Jenkins Automation:

- After the Jenkins setup, Click New Item -> Give Name as k8s-3-Tier-Arch -> Then click Pipeline -> Click 'Ok'.
- Inside that, there will be option for Configure in that paste the Pipeline code one by one save it, then build.

→ **Pipeline States:**

▪ Checkout:

```
pipeline {
    agent any

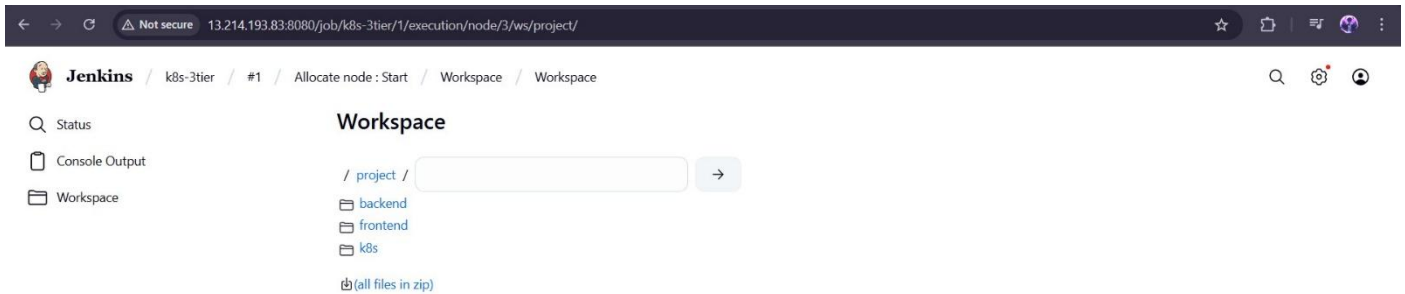
    environment {
        // DOCKERHUB_CREDENTIALS = credentials('dockerhub-cred') //
        Jenkins credential
        DOCKERHUB_USERNAME = 'prabathd'
        IMAGE_TAG = "${BUILD_NUMBER}"
    }
}
```

```

    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/prabath-ux/k8s-3-tier-arch.git'
            }
        }
    }
}

```

- The workspace will be created after success message.



→ Backend Image:

- Again, go to configure then paste the backend image code and save it then build.

```

stage('Build Backend Image') {
    steps {
        dir('project/backend') {
            sh "docker build -t $DOCKERHUB_USERNAME/backend-
abc:${IMAGE_TAG} ."
        }
    }
}

```

```
← → ↻ ⚠ Not secure 13.214.193.83:8080/job/k8s-3tier/3/console ☆ 📄 🔍 🗑️ 👤 ⋮

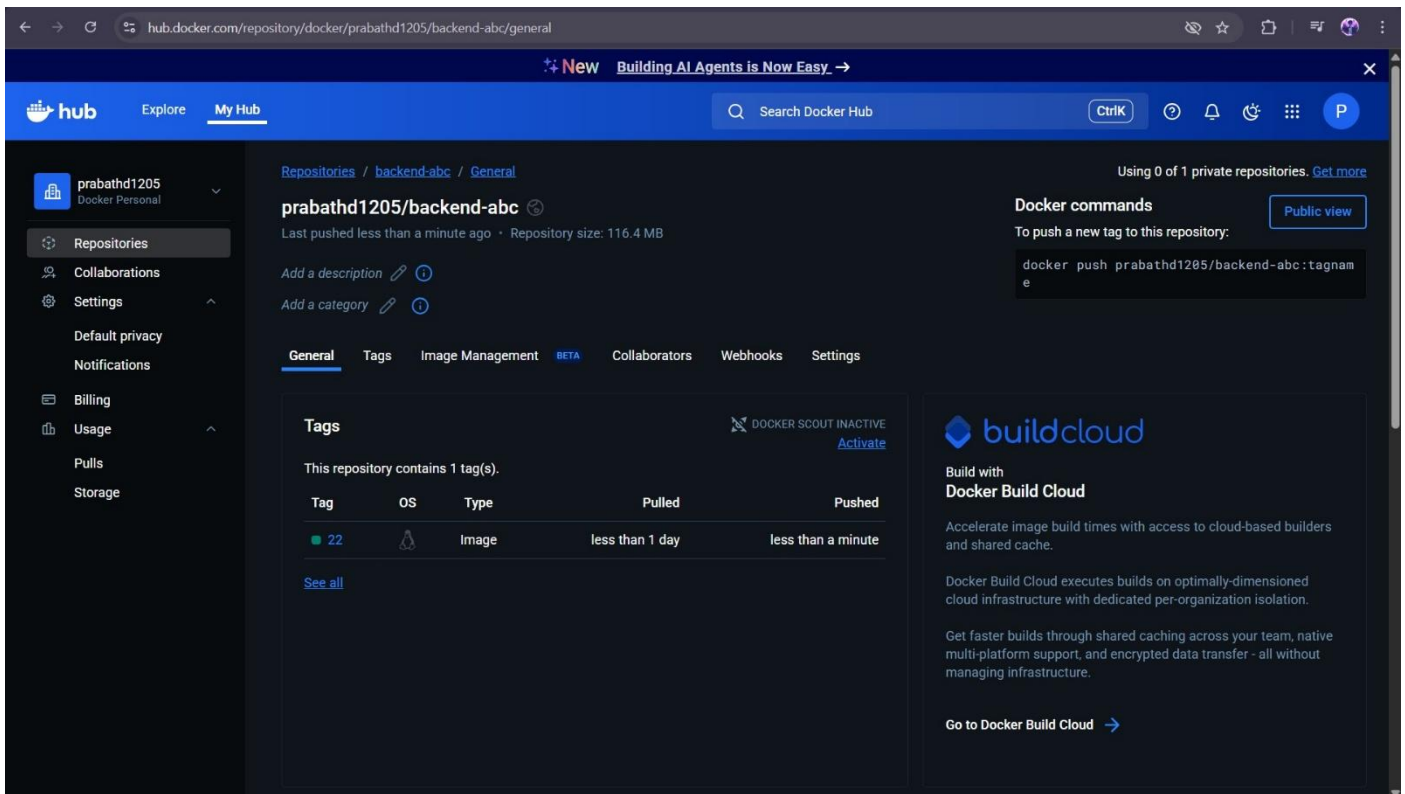
Jenkins / k8s-3tier / #3 🔍 ⚙️ 👤

Successfully installed bcrypt-4.3.0 blinker-1.9.0 click-8.2.1 flask-3.1.2 flask-cors-6.0.1 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.2
mysql-connector-python-9.4.0 werkzeug-3.1.3
@[91mWARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is
recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
@[0m@[91m
[notice] A new release of pip is available: 24.0 -> 25.2
[notice] To update, run: pip install --upgrade pip
@[0m ---> Removed intermediate container 02cc51bd9449
---> e028e4af207b
Step 5/6 : COPY src/ .
---> 5f5179d432a1
Step 6/6 : CMD ["python", "app.py"]
---> Running in b5178af23f9c
---> Removed intermediate container b5178af23f9c
---> 27d9759e1d57
Successfully built 27d9759e1d57
Successfully tagged prabathd/backend-abc:3
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

→ Push State:

- Go to Configure, paste the code for push state so that Docker image will appear in the Docker repository.
- But before that go to credentials, Click System(Global) -> Add Credentials -> Click Username and Password, it will have options for Secret Text.
- After Clicking the Secret text -> copy the PAT created in the Docker and paste it, then give the credentials as dockerpass.
- Then save it, and give build now option. It will push the image in the docker.

```
stage('Docker Push') {
    steps {
        script {
            withCredentials([string(credentialsId: 'dockerpass', variable:
            'dockerPassword')]) {
                sh "docker login -u $DOCKERHUB_USERNAME -p
                ${dockerPassword}"
                sh "docker push $DOCKERHUB_USERNAME/backend-
                123:${IMAGE_TAG}"
            }
        }
    }
}
```

→ Deploy State:

- Then go to configure again, put the deploy state code and then save it.
- Before that the Kubernetes triggers with Jenkins and Kubernetes CLI must be installed in the Plugins.
- Now, the pods will be created automatically for Kubernetes cluster, it can be viewed in the putty server.

```
stage('Deploy on k8s') {
    steps {
        script {
            withKubeCredentials(kubectlCredentials: [[ credentialsId:
'kubernetes' ]]) {
                sh "kubectl apply -f project/k8s/db-deployment.yaml"
                sh "kubectl apply -f project/k8s/backend-deployment.yaml"
                // sh "kubectl apply -f project/k8s/frontend-deployment.yaml"
                sh "kubectl set image deployment/backend
backend=$DOCKERHUB_USERNAME/backend-abc:${IMAGE_TAG}"
                sh "kubectl set image deployment/backend
// frontend=$DOCKERHUB_USERNAME/frontend-abc:${IMAGE_TAG}"
            }
        }
    }
}
```

```

root@ip-172-31-44-84:~/myproject# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-6d75c59d86-7f2k6           1/1     Running   0           8m56s
frontend-6b9d4f7658-jlnt2         1/1     Running   0           8m56s
mysql-d769b5ff8-h6l9k              1/1     Running   0           21m

```

→ Frontend Deployment:

- Just need to edit the pipeline codes to run the frontend image also.
- Then after the setting it, go to put server give the command as
 - > cd .kube
 - > vi config
 - > Whole text will be present copy all of it and save it as one text file, then go to credentials again in the Jenkins.
 - > Click add credentials -> Click Secret file -> Upload the saved text file.
 - > Then save and run the Pipeline now.

```
pipeline {
```

```
    agent any
```

```
    environment {
```

```
        // DOCKERHUB_CREDENTIALS = credentials('dockerhub-cred') //
        Jenkins credential
```

```
        DOCKERHUB_USERNAME = 'prabathd'
```

```
        IMAGE_TAG = "${BUILD_NUMBER}"
```

```
    }
```

```
    stages {
```

```
        stage('Checkout') {
```

```
            steps {
```

```
                git url: 'https://github.com/prabath-ux/k8s-3-tier-arch.git'
```

```
            }
```

```
        }
```

```
        stage('Build Backend Image') {
```

```
            steps {
```

```

        dir('project/backend') {
            sh "docker build -t $DOCKERHUB_USERNAME/backend-
abc:${IMAGE_TAG} ."
        }
    }
}

stage('Build Frontend Image') {
    steps {
        dir('project/frontend') {
            sh "docker build -t $DOCKERHUB_USERNAME/frontend-
abc:${IMAGE_TAG} ."
        }
    }
}

stage('Docker Push') {
    steps {
        script {
            withCredentials([string(credentialsId: 'dockerpass', variable:
'dockerPassword')]) {
                sh "docker login -u $DOCKERHUB_USERNAME -p
${dockerPassword}"

                sh "docker push $DOCKERHUB_USERNAME/backend-
123:${IMAGE_TAG}"

                sh "docker push $DOCKERHUB_USERNAME/frontend-
123:${IMAGE_TAG}"
            }
        }
    }
}

stage('Deploy on k8s') {
    steps {
        script {

```

```

withKubeCredentials(kubectlCredentials: [[ credentialsId:
'kubernetes' ]]) {

    sh "kubectl apply -f project/k8s/db-deployment.yaml"

    sh "kubectl apply -f project/k8s/backend-deployment.yaml"

    sh "kubectl apply -f project/k8s/frontend-deployment.yaml"


    // Set the new images for backend and frontend

    sh "kubectl set image deployment/backend
backend=$DOCKERHUB_USERNAME/backend-abc:${IMAGE_TAG}"

    sh "kubectl set image deployment/frontend
frontend=$DOCKERHUB_USERNAME/frontend-abc:${IMAGE_TAG}"

}

}

}

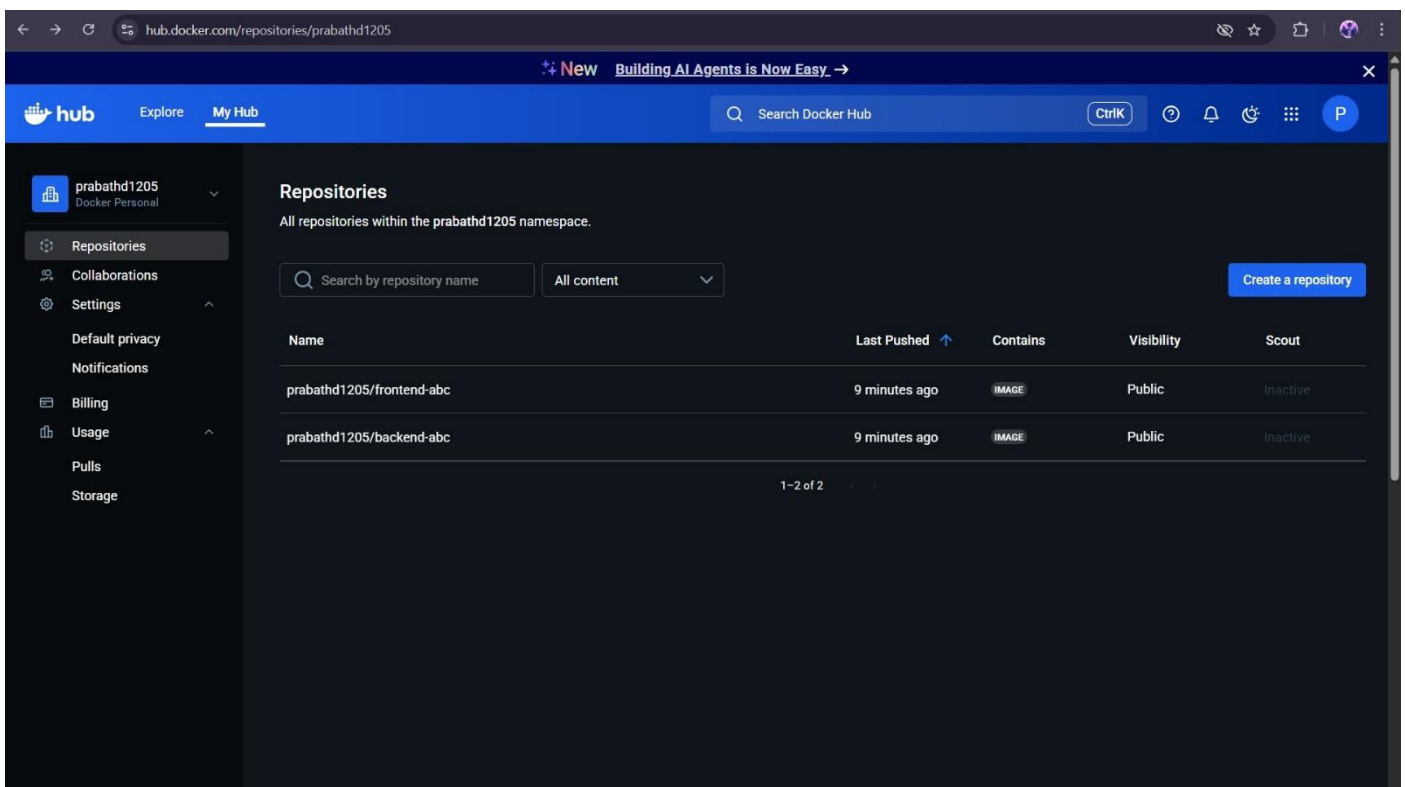
}

}

}

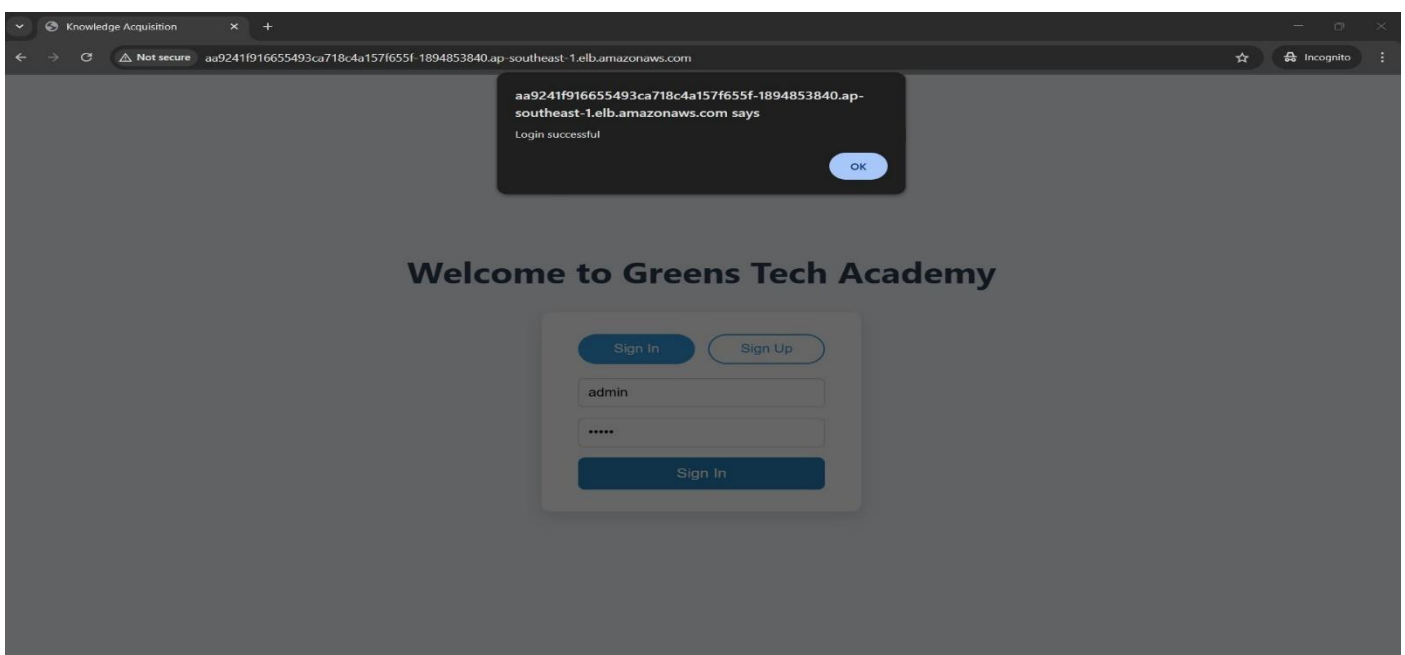
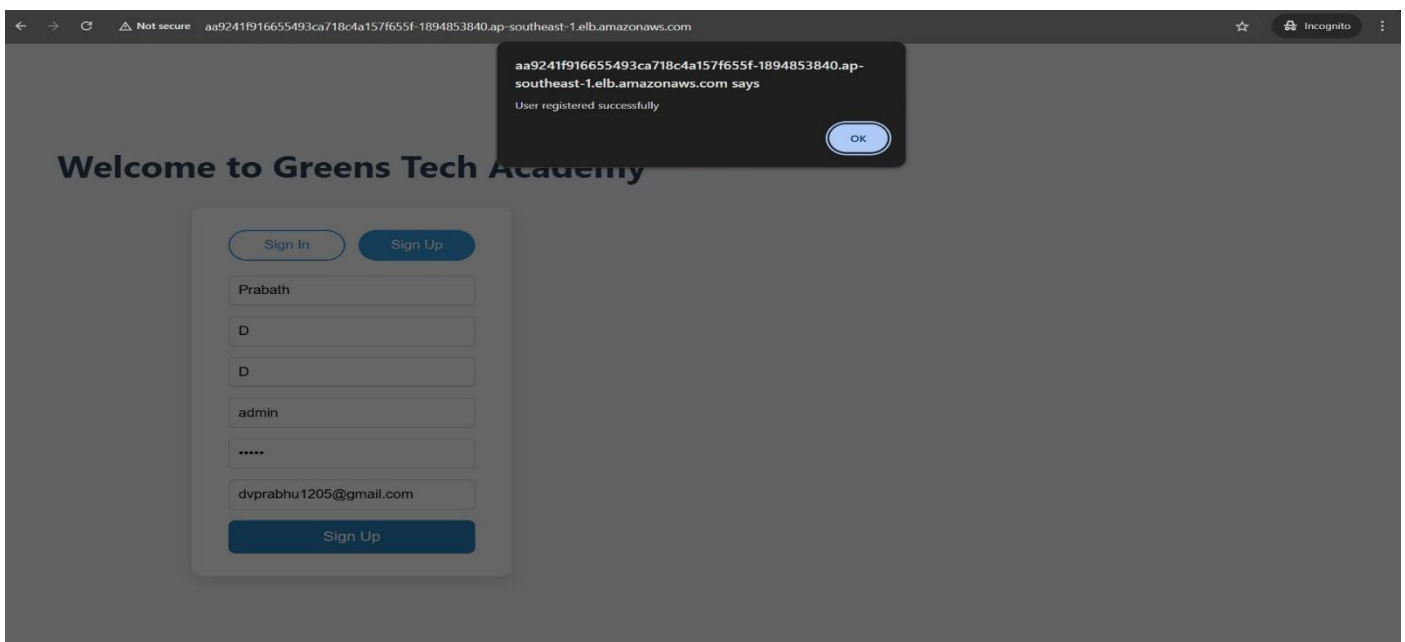
}

```



vii. 3-Tier Setup:

- After successful creation of Frontend and Backend Images, go to putty server and give the command 'kubectl get svc'.
- It will give the load balancer DNS name of the frontend copy that and edit the index.html, backend.html and student.html by pasting it in the specific lb-dns-name.
- After editing save it and paste that lb-dns in the web browser, it will run the Students Registration data form.
- After the setup of username and password to log in, give the student details.

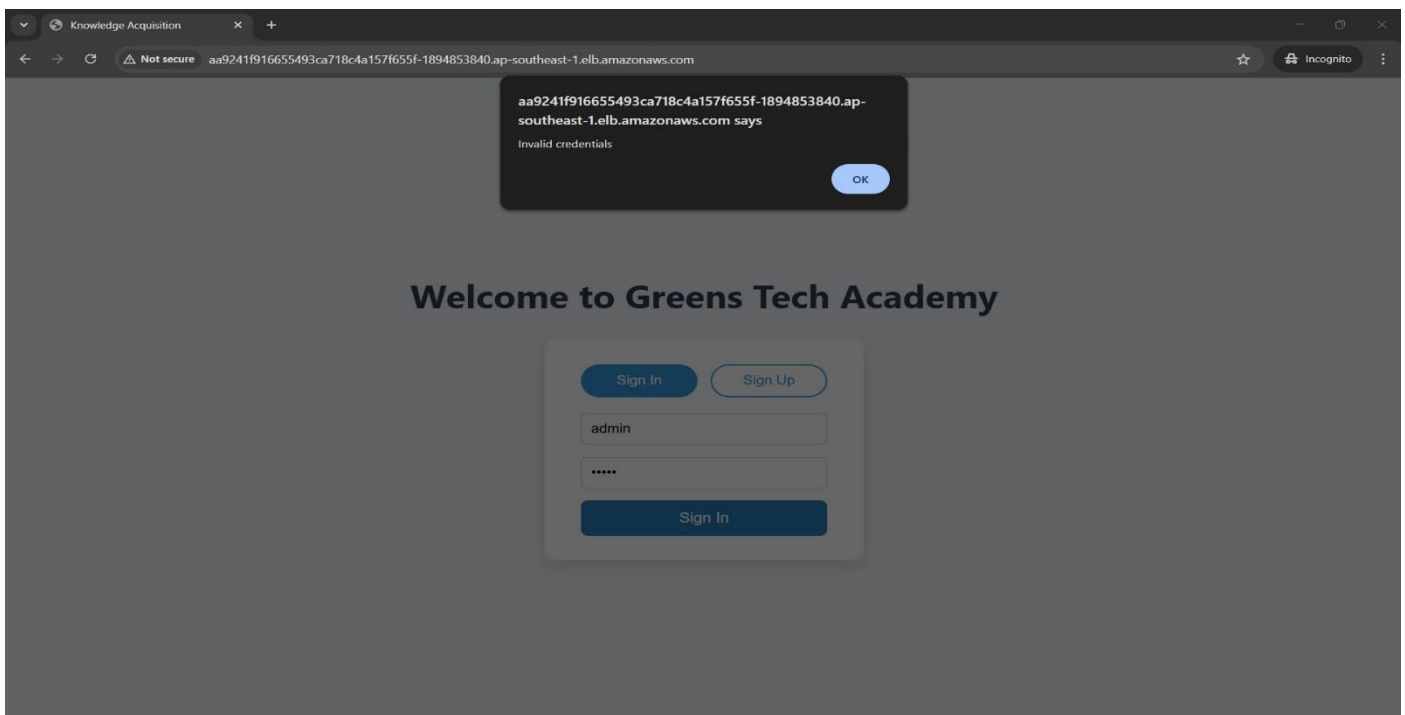


viii. Persistent Volume Claim:

- Now, in case if the pod gets deleted the data stored in the backend and in the database will also get deleted.
- So, in order to prevent that the database can be initiated with the Persistent Volume Claim.
- To test it, the pod can be deleted and add the new code inside the db-deployment.yaml file and save it by pushing it to the git hub.
- Then build again in the Jenkins it will show the success message, now even if the pods get deleted the data will be stored in the database.

```
root@ip-172-31-44-84:~/myproject# kubectl delete pod mysql-d769b5ff8-h6l9k
pod "mysql-d769b5ff8-h6l9k" deleted
root@ip-172-31-44-84:~/myproject# kubectl delete pod backend-6d75c59d86-7f2k6
pod "backend-6d75c59d86-7f2k6" deleted
root@ip-172-31-44-84:~/myproject# █
```

```
root@ip-172-31-44-84:~/myproject# kubectl delete pod mysql-75d6c585f8-95bnp
pod "mysql-75d6c585f8-95bnp" deleted
root@ip-172-31-44-84:~/myproject# kubectl delete pod backend-5588f755c5-7rkr9
Error from server (NotFound): pods "backend-5588f755c5-7rkr9" not found
root@ip-172-31-44-84:~/myproject# kubectl delete pod backend-5588f755c5-f8vp7
pod "backend-5588f755c5-f8vp7" deleted
root@ip-172-31-44-84:~/myproject# █
```



- It will show like this when the pods get deleted and try to give the existing credentials.

→ **db-deployment.yaml(PVC):**

apiVersion: v1

kind: ConfigMap

metadata:

name: mysql-initdb-config

data:

init.sql: |

CREATE DATABASE IF NOT EXISTS knowledge_acquisition;

USE knowledge_acquisition;

CREATE TABLE IF NOT EXISTS users (

id INT AUTO_INCREMENT PRIMARY KEY,

first_name VARCHAR(100),

middle_name VARCHAR(100),

last_name VARCHAR(100),

username VARCHAR(100) UNIQUE,

password VARCHAR(255),

email VARCHAR(100)

);

CREATE TABLE IF NOT EXISTS students (

id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(255),

date_of_joining DATE,

fees_paid DECIMAL(10,2),

department VARCHAR(100),

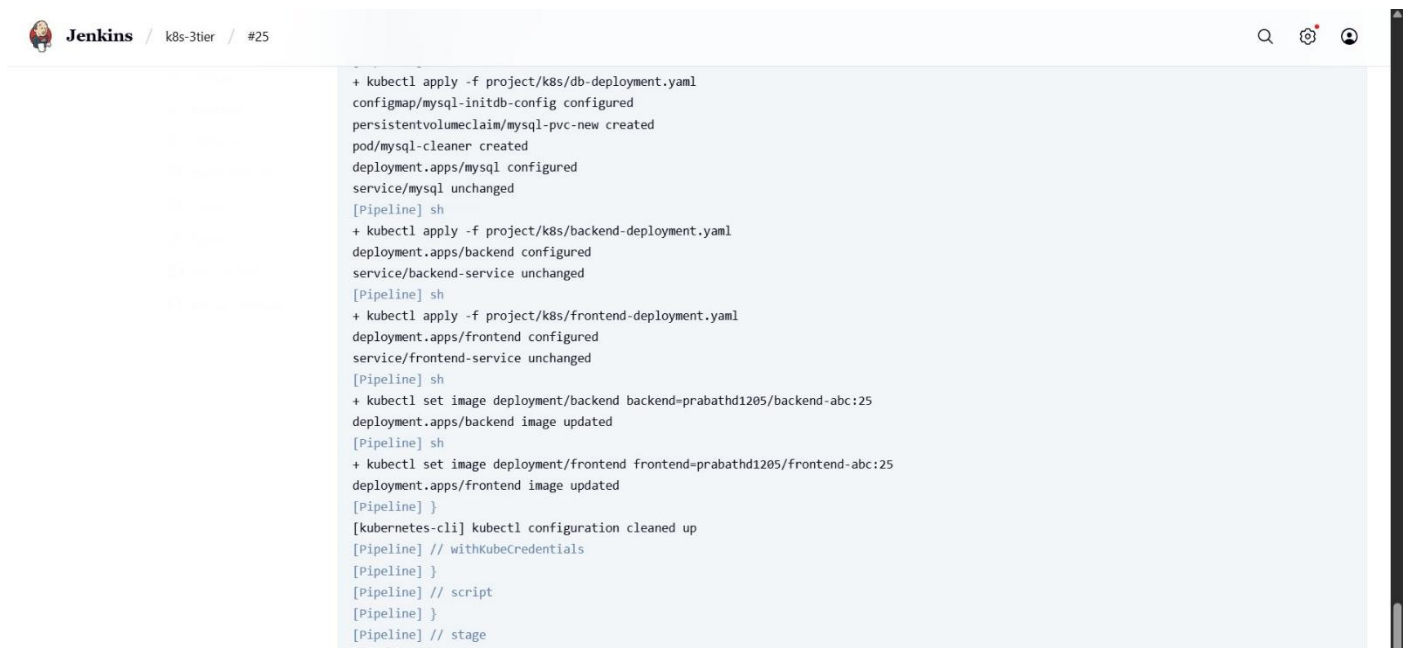
trainer_name VARCHAR(100),

```
        company_name VARCHAR(100)
    );
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc-new # NEW NAME
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: kops-csi-1-21
---
apiVersion: v1
kind: Pod
metadata:
  name: mysql-cleaner
spec:
  containers:
    - name: cleaner
      image: busybox
      command: [ "sh", "-c", "rm -rf /mnt/mysql/*; sleep 3600" ]
      volumeMounts:
        - name: mysql-data
          mountPath: /mnt/mysql
  volumes:
    - name: mysql-data
      persistentVolumeClaim:
```



```
    claimName: mysql-pvc-new
    restartPolicy: Never
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  replicas: 1
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: rootpassword
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
            - name: initdb
              mountPath: /docker-entrypoint-initdb.d # This is important
```

```
volumes:
- name: mysql-data
  persistentVolumeClaim:
    claimName: mysql-pvc-new
- name: initdb
  configMap:
    name: mysql-initdb-config
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  type: ClusterIP
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
```

A screenshot of a Jenkins console window. The top bar shows the Jenkins logo, the job name 'k8s-3tier', and the build number '#25'. The console output displays a series of commands and their results for deploying a MySQL database and its services. The commands use 'kubectl' to apply configurations, create resources, and update images. The output shows successful configurations for the MySQL database, its service, and the frontend/backend services.

```
+ kubectl apply -f project/k8s/db-deployment.yaml
configmap/mysql-initdb-config configured
persistentvolumeclaim/mysql-pvc-new created
pod/mysql-cleaner created
deployment.apps/mysql configured
service/mysql unchanged
[Pipeline] sh
+ kubectl apply -f project/k8s/backend-deployment.yaml
deployment.apps/backend configured
service/backend-service unchanged
[Pipeline] sh
+ kubectl apply -f project/k8s/frontend-deployment.yaml
deployment.apps/frontend configured
service/frontend-service unchanged
[Pipeline] sh
+ kubectl set image deployment/backend backend=prabathd1205/backend-abc:25
deployment.apps/backend image updated
[Pipeline] sh
+ kubectl set image deployment/frontend frontend=prabathd1205/frontend-abc:25
deployment.apps/frontend image updated
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
```

(Persistent Volume Claim)

```

root@ip-172-31-44-84:~/myproject# kubectl get pv
NAME                                CAPACITY  ACCESS MODES  RECLAIM POLICY
STATUS  CLAIM                                STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
pvc-5216874f-412c-4b55-9c1c-d8ea73adae0d  5Gi       RWO            Delete
Bound   default/mysql-pvc-new  kops-csi-1-21  <unset>
20m
root@ip-172-31-44-84:~/myproject# kubectl get pvc
NAME                                STATUS  VOLUME                                CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
mysql-pvc-new  Bound   pvc-5216874f-412c-4b55-9c1c-d8ea73adae0d  5Gi       RWO            kops-csi-1-21  <unset>
20m
root@ip-172-31-44-84:~/myproject# █

```

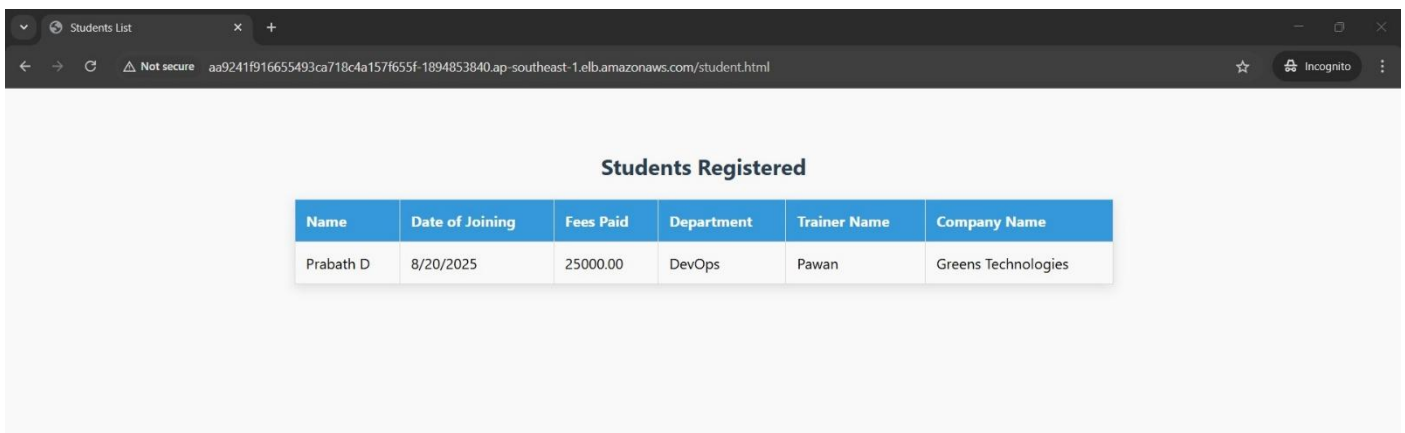
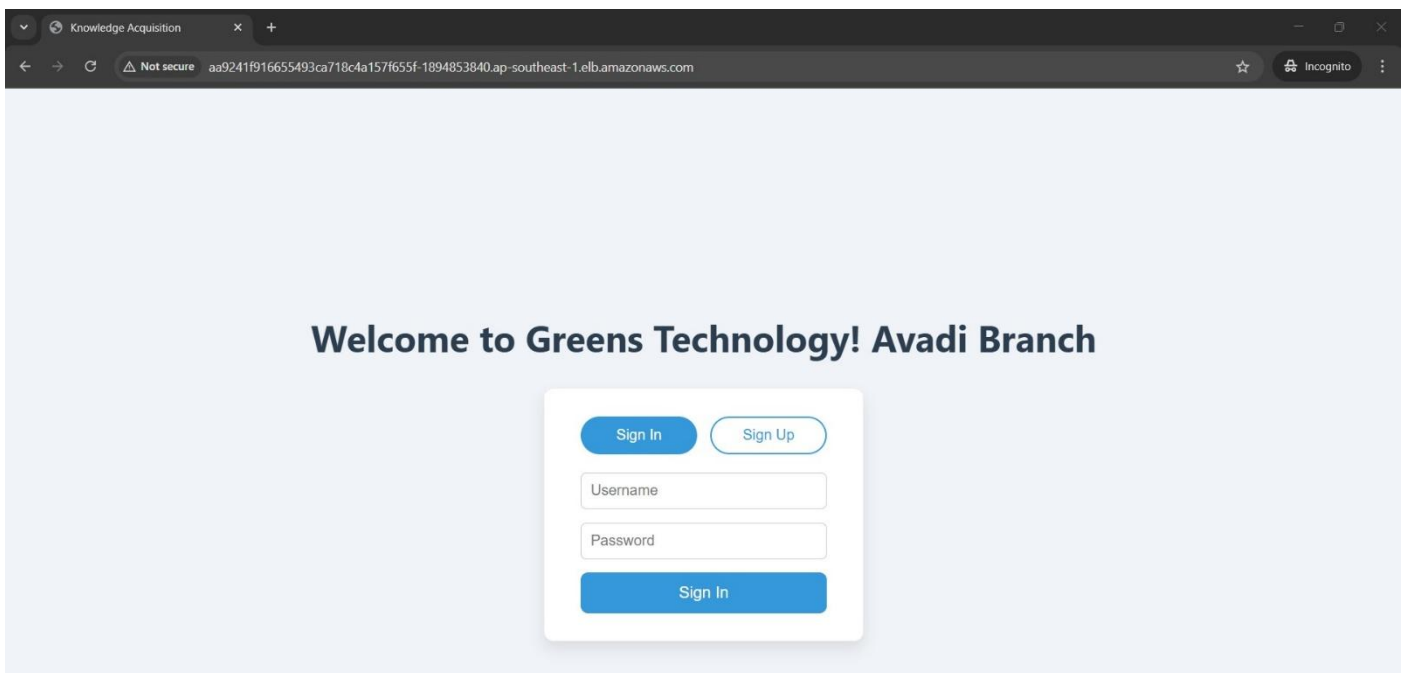
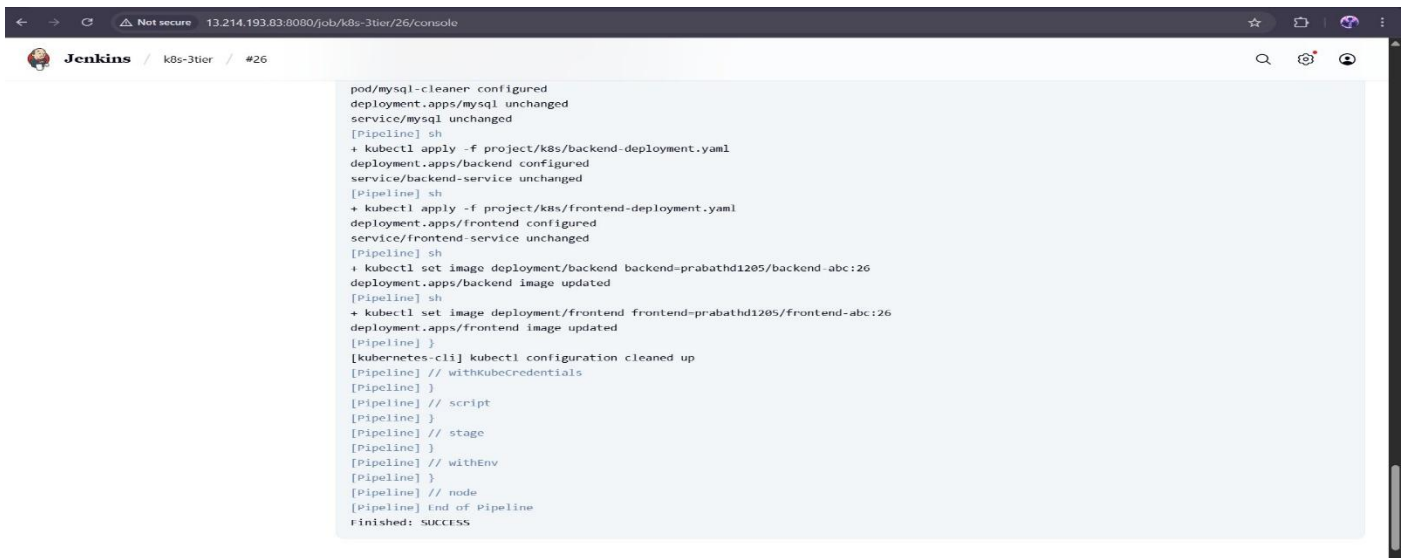
- The pods had been recreated now load the page. It will show the login successful message. Now the data will be stored in the database.

```

root@ip-172-31-44-84:~/myproject# kubectl get pods
NAME                                READY  STATUS   RESTARTS  AGE
backend-5588f755c5-7rkr9           1/1    Running  0          99s
frontend-7577c767f-c67ts           1/1    Running  0          98s
mysql-75d6c585f8-95bpn             1/1    Running  2 (91s ago) 100s
mysql-cleaner                       1/1    Running  0          100s
root@ip-172-31-44-84:~/myproject# kubectl delete pod backend-5588f755c5-7rkr9
pod "backend-5588f755c5-7rkr9" deleted
root@ip-172-31-44-84:~/myproject# kubectl get pods
NAME                                READY  STATUS   RESTARTS  AGE
backend-5588f755c5-f8vp7           1/1    Running  0          36s
frontend-7577c767f-c67ts           1/1    Running  0          3m20s
mysql-75d6c585f8-95bpn             1/1    Running  2 (3m13s ago) 3m22s
mysql-cleaner                       1/1    Running  0          3m22s
root@ip-172-31-44-84:~/myproject# █

```

- We can do another change also, go to index.html file and edit the “Greens Tech Academy” -> “Greens Tech Academy !! Avadi Branch”.
- Then save, push it to the git hub and add webhook for this change to happen and appear in the frontend page. After creation of webhook, the ping message be triggered successfully and then go to Jenkins -> Pipeline.
- In that there will be option for GitHub hook triggers for GITScm polling click that and save it. Then build the pipeline and verify the output.
- After building the file, the Jenkins will show SUCCESS message. Then you can view the changes happen in the frontend page of the Students Registration Form .



✦ With that, CI/CD Deploy on 3 – Tier Architecture with Kubernetes is implemented.